



MediCore HMS

Hospital Management System

Comprehensive Technical & Functional Documentation
covering the Mobile App and Web Platform

2

Platforms

28

Screens

16

Data Models

4

User Roles

React Native

Expo SDK 55

Vanilla JS

LocalStorage

AsyncStorage

RBAC

HIPAA-Ready

Version 1.0.0 · July 2026 · MediCore General Hospital

Table of Contents

OVERVIEW

1. Project Overview	3
2. Technology Stack	3
3. System Architecture	4

MOBILE APPLICATION (REACT NATIVE)

4. App Overview & Navigation	5
5. Authentication & User Roles	5
6. Screen Reference — All 28 Screens	6
7. App State Management	9
8. App Components Library	9

WEB PLATFORM

9. Website Overview	10
10. Landing Page Sections	10
11. Web Dashboard Pages	11
12. Web Navigation & RBAC	12
13. Web Data Layer	12

DATA & MODELS

14. Data Models Reference	13
15. Status & Badge Reference	15

SECURITY & OPERATIONS

16. Role-Based Access Control	16
17. Demo Credentials	16

18. Deployment & Setup Guide	17
19. Known Limitations	17

1. Project Overview

MediCore HMS (Hospital Management System) is a full-featured healthcare management platform delivered across two parallel implementations: a **React Native mobile app** (powered by Expo) and a **Vanilla JS web application**. Both platforms share identical data models, the same seed dataset, and equivalent functionality — giving hospital staff a consistent experience whether they access the system from a smartphone or a desktop browser.

The system covers every major hospital workflow: patient registration, OPD queuing, appointments, clinical admissions, nursing notes, laboratory & radiology orders, pharmacy dispensing, billing, inventory management, staff management, reporting, emergency triage, real-time messaging, and role-based access control.



MOBILE APP

**React Native · Expo
SDK 55**



WEB PLATFORM

**HTML5 · CSS3 ·
Vanilla JS**



HOSPITAL NAME

**MediCore General
Hospital**



AUTH MODEL

Role-Based (4 roles)



STORAGE (APP)

**AsyncStorage (device-
local)**



STORAGE (WEB)

**localStorage
(browser-local)**

i Both platforms are **offline-first**. All data is persisted locally using AsyncStorage (app) or localStorage (web). There is no backend server or database required — making this ideal for demos, prototyping, and local deployments.

2. Technology Stack

Mobile App Stack

Package	Version	Purpose
<code>expo</code>	~55.0.0	Build toolchain, dev server, OTA updates
<code>react</code>	19.2.0	UI library
<code>react-native</code>	0.83.6	Cross-platform mobile framework
<code>@react-navigation/native</code>	^7.0.0	Navigation container & core
<code>@react-navigation/native-stack</code>	^7.0.0	Stack navigator (Auth / Detail screens)
<code>@react-navigation/drawer</code>	^7.0.0	Sidebar drawer navigator
<code>@react-navigation/bottom-tabs</code>	^7.0.0	Bottom tab bar
<code>@expo/vector-icons</code>	^15.0.2	Ionicons icon set
<code>expo-linear-gradient</code>	~55.0.15	Gradient backgrounds
<code>@react-native-async-storage/async-storage</code>	2.2.0	Persistent local storage
<code>react-native-gesture-handler</code>	~2.30.0	Gesture support for drawer
<code>react-native-reanimated</code>	4.2.1	Animation engine
<code>react-native-screens</code>	~4.23.0	Native screen containers
<code>expo-splash-screen</code>	~55.0.22	Splash screen management
<code>expo-status-bar</code>	~55.0.6	Status bar control (dark/light mode)

Web Platform Stack

Technology	Purpose
HTML5	Semantic page structure, multi-page app (MPA)
CSS3 (custom)	<code>landing.css</code> for landing page, <code>style.css</code> for dashboard pages
Vanilla JavaScript	DOM manipulation, data layer, navigation RBAC, page logic
Google Fonts — Outfit	Typography (weights 300–900)
Font Awesome 6.5	Icon set via CDN
localStorage API	Client-side data persistence
sessionStorage API	Logged-in user session

3. System Architecture

App Architecture

```
// App Entry Point App.js └─ GestureHandlerRootView └─ AppProvider (Context +
AsyncStorage) └─ AppNavigator └─ NavigationContainer └─ RootStack (Stack Navigator) └─
[Unauthenticated] └─ LoginScreen └─ RegisterScreen └─ [Authenticated] └─ MainDrawer
(Drawer Navigator – 22 screens) └─ HomeTabs (Bottom Tab Navigator – 5 tabs) └─
DashboardScreen └─ ReportsScreen └─ AppointmentsScreen (x2 tabs) └─ SettingsScreen
└─ PatientProfileScreen └─ DoctorProfileScreen └─ AddAppointmentScreen └─
AddPatientScreen
```

Web Architecture

```
website/ └─ index.html ← Landing page (public) └─ css/ └─ landing.css ← Landing
page styles └─ style.css ← Dashboard page styles (shared) └─ js/ └─ data.js ←
HMSData singleton (localStorage CRUD) └─ nav.js ← Sidebar nav renderer + RBAC guard └─
app.js ← Shared utilities, auth helpers, toast └─ main.js ← Landing page
interactions └─ pages/ ← 28 protected dashboard pages └─ login.html / register.html
└─ dashboard.html └─ [26 module pages]
```

State & Data Flow

Platform	State Mechanism	Persistence Key	Session Key
Mobile App	React Context + useReducer	<code>hms_data_v1</code> (AsyncStorage)	<code>hms_user</code> (AsyncStorage)
Web	HMSData singleton object	<code>hms_data_v1</code> (localStorage)	<code>hms_user</code> (sessionStorage)

✓ The same storage key `hms_data_v1` and identical SEED data are used on both platforms, making it easy to reason about the data shape regardless of which platform you're working on.

4. App Navigation Structure

The app uses a three-layer navigation hierarchy:

- **RootStack** — Switches between the auth flow and the main app based on `state.user`
- **MainDrawer** — Dark sidebar (background #0f172a) with 3 sections: Main, Clinical, Administration. Width: 270px.
- **HomeTabs** — Custom bottom tab bar with 5 tabs including a center "Book Test" FAB button that navigates to Laboratory

Sidebar Navigation Sections

Section	Items	Notes
Main	Dashboard, OPD Queue, Patients, Doctors, Departments	All roles
Clinical	Admissions, Discharge, Medical Records, Nursing, Laboratory, Radiology, Pharmacy, Rooms & Wards, Emergency	Emergency shows critical patient badge
Administration	Staff, Inventory, Billing, Reports, User Management, Messages, Notifications, Settings	User Management: Administrator only. Messages shows unread badge.

Bottom Tab Bar

Tab	Icon	Screen	Note
Home	home	DashboardScreen	—
Reports	document-text	ReportsScreen	—
Book Test	add (FAB)	LaboratoryScreen	Center elevated button, blue (#11B5E4)
Appointments	calendar	AppointmentsScreen	—
Profile	person	SettingsScreen	—

5. Authentication & User Roles

Login Flow

Both platforms use a credential-based login. The app reads registered users from `hms_registered_users` (AsyncStorage). The web uses `hms_registered_users` (localStorage). On first launch, no users exist — users must register or use demo credentials.

Demo Credentials

Email	Password	Role	Name
admin@medicore.com	admin123	Administrator	Dr. Admin
doctor@medicore.com	doctor123	Doctor	Dr. Mike Chen
nurse@medicore.com	nurse123	Nurse	Nurse Emily
staff@medicore.com	staff123	Staff	John Staff

Role-Based Access Control (RBAC)

Module	Administrator	Doctor	Nurse	Staff
Dashboard	✓	✓	✓	✓
Patients	✓	✓	✓	✓
Doctors	✓	✓	✓	✓
Departments	✓	✓	✓	✓
OPD Queue	✓	—	✓	✓
Admissions	✓	✓	✓	✓
Discharge	✓	✓	✓	✓
Medical Records	✓	✓	✓	✓
Nursing	✓	✓	✓	—
Laboratory	✓	✓	✓	✓
Radiology	✓	✓	✓	✓
Pharmacy	✓	✓	✓	✓
Rooms & Wards	✓	✓	✓	✓
Emergency	✓	✓	✓	✓
Staff	✓	✓	✓	✓
Inventory	✓	—	—	✓
Billing	✓	✓	✓	✓
Reports	✓	✓	—	✓
User Management	✓	—	—	—
Messages	✓	✓	✓	✓
Notifications	✓	✓	✓	✓
Settings	✓	✓	✓	✓

✓ = Full access · — = Hidden or redirected · Web enforces via `nav.js guardPage()`; App enforces via sidebar filter.

6. Screen Reference — Mobile App

Authentication Screens



LoginScreen

All users

- Email + password inputs with show/hide toggle
- Gradient hero banner with stats (1,240+ patients, 48 doctors)
- Reads from hms_registered_users AsyncStorage key
- Navigate to RegisterScreen



RegisterScreen

New users

- Name, email, password, role selection
- Saves new user to hms_registered_users array
- Auto-redirects to Login after success

Main Module Screens



DashboardScreen

All roles

- Hero image banner with "Your Health, Our Priority"
- 4 feature pills: Reports, Safe & Secure, Fast Delivery, 24/7 Support
- Quick Actions grid (Book Test, My Reports, Appointments, Find Center)
- Popular Services horizontal scroll (Blood, Genetic, Cardiac, Covid-19 tests)
- Promo banner with "Flat 20% OFF on First Test"
- Unread notification badge on bell icon



PatientsScreen

All roles

- Searchable & filterable patient list
- Patient cards with status badges (Stable, Critical, Admitted, etc.)
- Quick stats bar: total, admitted, critical
- FAB button → AddPatientScreen
- Tap patient → PatientProfileScreen



PatientProfileScreen

All roles

- Demographics, contact, blood group, assigned doctor
- Live vitals: BP, HR, Temperature, SpO2
- Medical conditions & allergy list
- Visit count, admission count, total billed



DoctorsScreen

All roles

- Doctor cards with specialty, rating, patient count
- Real-time status badge (Available, In Surgery, On Leave)
- Search by name or specialty
- Tap → DoctorProfileScreen

- Action buttons: Book Appointment, View Records



AppointmentsScreen

All roles

- Date-filterable appointment list
- Type badges: Checkup, Consultation, Follow-up
- Status filters: Confirmed, Pending, Checked In, Cancelled
- FAB → AddAppointmentScreen



DepartmentsScreen

All roles

- Department cards: head doctor, doctor count, bed occupancy
- 6 departments: Cardiology, Neurology, Orthopedics, Pediatrics, Dermatology, Oncology
- Occupancy progress bars

Clinical Screens



AdmissionsScreen

All roles

- Active inpatient list with ward, room, bed assignment
- Admission date, attending doctor, diagnosis
- Status: Admitted, Critical



DischargeScreen

All roles

- Process patient discharge from inpatient ward
- Discharge summary, final billing link



OPDQueueScreen

Admin, Nurse, Staff

- Token-numbered queue for outpatient department
- Per-patient: department, assigned doctor, wait time
- Status: Waiting, In Consultation, Completed



EmergencyScreen

All roles

- Active emergency cases with severity: Critical, Urgent
- Arrival time, assigned doctor, treatment status
- Red badge on sidebar nav showing critical count



MedicalRecordsScreen

All roles

- Patient-linked record list by date
- Record types: Checkup, Emergency, Follow-up
- Doctor notes, diagnosis text



NursingScreen

Admin, Doctor, Nurse

- Nursing notes with vitals snapshot
- Note types: Vitals, Medication
- Nurse name, timestamp, patient link



LaboratoryScreen

All roles

- Lab orders list with test name, ordering doctor
- Priority badges: Routine, Urgent



RadiologyScreen

All roles

- Radiology orders: X-Ray, CT, MRI
- Priority and status tracking same as Lab

- Status: Pending, In Progress, Completed



PharmacyScreen

All roles

- Prescription list with medicine name, quantity
- Prescribing doctor, patient link
- Status: Dispensed, Pending



RoomsScreen

All roles

- Room/bed inventory per ward
- Types: General, ICU, Emergency
- Occupancy status: Available, Occupied

Administration Screens



StaffScreen

All roles

- Staff list with role, department, shift
- Status: Active, Inactive



InventoryScreen

Admin, Staff

- Medical supplies with quantity & reorder level
- Status: In Stock, Low Stock, Out of Stock



BillingScreen

All roles

- Invoice list with amount, paid amount, balance
- Status: Paid, Partial, Pending



ReportsScreen

Admin, Doctor, Staff

- Department performance summaries
- Patient statistics, financial overview



MessagesScreen

All roles

- Internal messages with read/unread state
- Unread count shown in sidebar badge



NotificationsScreen

All roles

- Types: danger (critical alert), info, warning
- Unread dot on header bell icon



UsersScreen

Administrator only

- User account management
- Role assignment, last login tracking



SettingsScreen

All roles

- Dark mode toggle
- Hospital name, profile info
- Logout action

7. App State Management

The app uses React Context + useReducer for global state, persisted via AsyncStorage. The AppProvider component wraps the entire app and exposes all CRUD helpers via context.

Context API Surface

Method / Property	Type	Description
<code>state.data</code>	Object	Full dataset (all 16 collections loaded from AsyncStorage)
<code>state.user</code>	Object null	Currently logged-in user: <code>{ name, email, role }</code>
<code>state.darkMode</code>	Boolean	Dark mode preference
<code>state.loading</code>	Boolean	True while AsyncStorage is being read on startup
<code>login(user)</code>	async fn	Persists user to AsyncStorage, sets <code>state.user</code>
<code>logout()</code>	async fn	Removes user key, clears <code>state.user</code>
<code>setData(key, value)</code>	async fn	Replaces an entire collection and saves to AsyncStorage
<code>addItem(key, item)</code>	async fn	Appends an item to a collection array
<code>updateItem(key, id, updates)</code>	async fn	Merges updates into the matching item (by <code>id</code>)
<code>removeItem(key, id)</code>	async fn	Filters out item with matching <code>id</code>
<code>toggleDark()</code>	async fn	Toggles dark mode, persists to <code>hms_dark</code> key

Reducer Actions

Action Type	Payload	Effect
INIT	data, user, darkMode	Sets all initial state, sets loading = false
LOGIN	user	Sets state.user
LOGOUT	—	Clears state.user to null
SET_DATA	key, value	Updates one key in state.data
TOGGLE_DARK	—	Flips state.darkMode boolean

AsyncStorage Keys

Key	Content	Notes
hms_data_v1	Full JSON dataset	Seeded from SEED on first launch
hms_user	Logged-in user object	Cleared on logout
hms_registered_users	Array of registered accounts	Used by LoginScreen for auth
hms_dark	"true" or "false" string	Dark mode preference

8. Shared Components Library

All shared UI primitives live in `src/components/` and are used across screens.

Component	File	Props	Purpose
Avatar	Avatar.js	name, size, color	Renders initials-based circular avatar with background color
Card	Card.js	style, children	White rounded card with shadow — base container for most list items
EmptyState	EmptyState.js	icon, title, subtitle	Centered empty-list illustration with icon, title and message
PageHeader	PageHeader.js	title, subtitle, action	Top section heading with optional right-side action button
ScreenHeader	ScreenHeader.js	title, onBack, rightAction	Navigation header with back button for stack screens
SearchBar	SearchBar.js	value, onChangeText, placeholder	Styled search input with magnifier icon
StatCard	StatCard.js	icon, label, value, color	Compact stat tile used in list page headers (e.g., total patients, critical count)
StatusBadge	StatusBadge.js	status	Pill badge with color from StatusColors map (e.g., Stable=green, Critical=red)

i **StatusBadge** uses the [StatusColors](#) map from `constants/colors.js` to automatically resolve background and text color for any status string — no per-screen styling needed.

9. Website Overview

The web platform is a Multi-Page Application (MPA) built with plain HTML5, CSS3, and Vanilla JavaScript. It shares the same data model as the mobile app. The site has two tiers:

- **Landing Page** (`index.html`) — Public marketing page with features, doctors showcase, appointment form, and sign-in CTA
- **Dashboard Pages** (`pages/*.html`) — 28 protected application pages requiring login, sharing a common sidebar layout via `nav.js`



ENTRY POINT

`index.html (Landing)`



AUTH ENTRY

`pages/login.html`



POST-LOGIN

`pages/dashboard.html`



TOTAL PAGES

29 HTML files



STYLESHEETS

**`landing.css +
style.css`**



JS MODULES

**`data.js · nav.js · app.js
· main.js`**

10. Landing Page Sections

The landing page (`index.html`) is a single-scroll marketing page with 9 distinct sections and animated scroll-reveal effects.

#	Section ID	Content	Key Features
1	#home — Navbar	Fixed top navigation bar	Logo, nav links (Features, Services, Doctors, Appointment, Contact), Sign In + Get Started buttons, hamburger for mobile
2	#home — Hero	Full-viewport hero section	Animated particles background, floating ambient cards (Appointment Confirmed, 148 Doctors Active), live stats glass card (2,847 patients, 148 doctors, 312 appointments), trust badges (HIPAA, Role-Based Access, 24/7 Uptime, Mobile Ready)
3	Stats Strip	Animated counter row	4 counters: 2,847 Active Patients, 148 Doctors, 312 Appointments Today, 8 Departments — animate on scroll via IntersectionObserver
4	#features	6-card feature grid	Smart Appointments, Digital Medical Records, Emergency Triage, Pharmacy & Inventory, Billing & Invoicing, Analytics & Reports
5	#about	About section, 2-col layout	Medical team image, 12+ Years badge, 5 bullet points on compliance and capabilities, Access Dashboard CTA
6	#services	6-card services/modules grid	Patient Management, Laboratory, Rooms & Wards, Staff Management, Internal Messaging, Security & Compliance
7	#doctors	4-card doctor showcase	Dr. Mike Chen (Cardiology), Dr. Lisa Park (Neurology), Dr. Anna White (Pediatrics), Dr. Rachel Green (Emergency) — each with rating, patients, experience
8	Testimonials	3 user testimonials	Quotes from a cardiologist, emergency doctor, and hospital administrator
9	#appointment	Appointment booking form	Left: working hours, emergency hotline, location. Right: form with name, phone, email, department select, date, notes. Redirects to login for full management.
10	FAQ	Accordion FAQ list	What is MediCore, data security, user access, supported devices, billing export — accordion expand/collapse
11	#contact — Footer	Footer with contact info	Address (1200 Medical Center Dr, Chicago IL), phone, email, social links, copyright

Landing Page JavaScript

File	Responsibilities
<code>main.js</code>	Navbar scroll shrink, hamburger menu toggle, animated stat counters (IntersectionObserver), scroll-reveal animations, FAQ accordion, particle generator, ripple effect on CTA button, appointment form toast notification

11. Web Dashboard Pages

All 28 dashboard pages live in `pages/` and share the same two-column layout: a fixed sidebar (rendered by `nav.js`) and a main content area. Each page loads `data.js`, `nav.js`, and `app.js`.

File	Module	Key Features
<code>login.html</code>	Authentication	Email + password form, role display, link to register. Saves user to sessionStorage on success.
<code>register.html</code>	Authentication	Name, email, password, role select. Saves to <code>hms_registered_users</code> in localStorage.
<code>dashboard.html</code>	Main	Summary stats: total patients, admissions, appointments, revenue. Recent activity feed. Quick-access shortcut cards.
<code>patients.html</code>	Main	Searchable patient table with status badges. Add patient button. Pagination. Click → <code>patient-profile.html</code>
<code>patient-profile.html</code>	Main	Full patient detail: demographics, vitals, conditions, allergies, visit history, billing summary, linked appointments.
<code>doctors.html</code>	Main	Doctor cards grid with specialty, rating, status. Search. Click → <code>doctor-profile.html</code>
<code>doctor-profile.html</code>	Main	Doctor bio, experience, schedule, patient count, department, contact info.
<code>departments.html</code>	Main	Department list with head, doctor count, bed total, occupancy bar chart.
<code>appointments.html</code>	Main	Date-filterable appointment table. Status filter tabs. Link to <code>add-appointment.html</code>
<code>add-appointment.html</code>	Main	Form: patient select, doctor select, department, date, time, appointment type. Saves via HMSData.add().
<code>opd-queue.html</code>	Main	Token-numbered queue table. Status update buttons (Waiting → In Consultation → Completed). Roles: Admin, Nurse, Staff.
<code>admissions.html</code>	Clinical	Active inpatient list: ward, room, bed, diagnosis, doctor, admission date. Status badges.
<code>discharge.html</code>	Clinical	Process patient discharge with summary notes and final billing trigger.
<code>medical-records.html</code>	Clinical	Filterable records by patient or date. Record types: Checkup, Emergency, Follow-up. Notes display.
<code>nursing.html</code>	Clinical	Nursing notes with vitals snapshot. Type: Vitals, Medication. Add new note form. Roles: Admin, Doctor, Nurse.
<code>laboratory.html</code>	Clinical	Lab order list. Priority badges (Routine/Urgent). Status workflow. Add new order form.

File	Module	Key Features
radiology.html	Clinical	Radiology orders: X-Ray, CT, MRI. Same structure as laboratory. Priority + status tracking.
pharmacy.html	Clinical	Prescription list. Dispense action. Medicine name, quantity, prescribing doctor, patient link.
rooms.html	Clinical	Room/bed inventory with ward, type (General/ICU/Emergency), occupancy. Availability status.
emergency.html	Clinical	Active emergency cases. Severity color-coded (Critical=red, Urgent=amber). Arrival time, assigned doctor.
staff.html	Administration	Staff table with role, department, shift, status. Add/edit staff.
inventory.html	Administration	Supply list with quantity, reorder level, status. Low Stock auto-flagged. Roles: Admin, Staff.
billing.html	Administration	Invoice list: amount, paid, balance, status (Paid/Partial/Pending). Revenue summary header.
reports.html	Administration	Department performance, patient stats, financial overview, staff analytics. Export-ready layout. Roles: Admin, Doctor, Staff.
users.html	Administration	User account management: name, email, role, department, last login, status. Admin only.
messages.html	Administration	Internal message list. Read/unread state. From/to/time display. Mark as read action.
notifications.html	Administration	Notification list by type: danger, info, warning. Mark all read. Timestamps.
settings.html	Administration	Hospital name config, dark mode toggle, account info, logout.
my-profile.html	Administration	Current user profile view: name, role, department, email.

12. Web Navigation & RBAC

The `nav.js` module runs on every dashboard page. It renders the sidebar dynamically based on the current user's role from `sessionStorage`, and guards pages against unauthorized access.

nav.js Functions

Function	Description
<code>getUserRole()</code>	Reads <code>sessionStorage.hms_user</code> , returns role string. Defaults to "Administrator" if missing.
<code>canAccess(item, role)</code>	Returns true if item has no role restriction, or if the user's role is in the item's <code>roles</code> array.
<code>renderNav()</code>	Builds and injects the full sidebar HTML into <code>.sidebar-nav</code> element. Filters items by role. Adds active class to current page. Renders live badges (appointment count, critical count, unread messages).
<code>guardPage()</code>	Checks if the current page URL requires a specific role. If user lacks access, shows a toast and redirects to <code>dashboard.html</code> .

Session Management (Web)

Storage	Key	Value	Lifecycle
sessionStorage	<code>hms_user</code>	<code>{ name, email, role }</code>	Set on login, cleared when browser tab closes
localStorage	<code>hms_data_v1</code>	Full dataset JSON	Persists across sessions. Seeded from SEED on first visit.
localStorage	<code>hms_registered_users</code>	Array of user accounts	Grows as users register

 The web platform uses **sessionStorage** for the user session — closing the browser tab will log the user out. This is by design for security in shared environments. The mobile app uses **AsyncStorage** which persists across app restarts.

13. Web Data Layer (HMSTData)

`data.js` exposes a global singleton `window.HMSTData` that provides all CRUD operations over `localStorage`. Every dashboard page uses this API exclusively — no `fetch/XHR` calls, no backend.

HMSTData API

Method	Signature	Description
<code>getAll()</code>	<code>() → Object</code>	Returns the full dataset object from <code>localStorage</code>
<code>get(key)</code>	<code>(key) → Array Object</code>	Returns a single collection by key (e.g., <code>'patients'</code>)
<code>set(key, value)</code>	<code>(key, value) → value</code>	Replaces an entire collection and saves
<code>add(key, item)</code>	<code>(key, item) → item</code>	Appends item to collection array and saves
<code>update(key, id, updates, idField?)</code>	<code>→ updatedItem null</code>	Merges updates into matching item. Default <code>idField</code> is <code>'id'</code> .
<code>remove(key, id, idField?)</code>	<code>→ void</code>	Removes item with matching <code>id</code> from collection
<code>find(key, id, idField?)</code>	<code>→ item null</code>	Returns first item matching <code>id</code> , or <code>null</code>
<code>nextId(prefix, key)</code>	<code>(str, str) → string</code>	Auto-generates next sequential ID. E.g., <code>nextId('P-', 'patients') → 'P-007'</code>
<code>getPatientName(p)</code>	<code>(patient) → string</code>	Returns <code>"firstName lastName"</code>
<code>getAge(dob)</code>	<code>(dateStr) → number</code>	Calculates age from ISO date string
<code>formatDate(d)</code>	<code>(dateStr) → string</code>	Returns locale-formatted date like "Jun 29, 2026"
<code>badgeForStatus(status)</code>	<code>(str) → cssClass</code>	Maps status string to a CSS badge class (e.g., <code>'badge-success'</code> , <code>'badge-danger'</code>)
<code>avatarUrl(name)</code>	<code>(str) → url</code>	Returns a <code>ui-avatars.com</code> URL for the given name
<code>reset()</code>	<code>() → data</code>	Clears <code>localStorage</code> and reloads from SEED data

```
// Example usage on any dashboard page
const patients = HMSData.get('patients');
const newPatient = { id: HMSData.nextId('P-', 'patients'), firstName: 'Jane', ... };
HMSData.add('patients', newPatient);
HMSData.update('appointments', 'A-1001', { status: 'Completed' });
HMSData.remove('staff', 'S-003');
```

14. Data Models Reference

All 16 data collections are stored under `hms_data_v1`. Below are the full field schemas for each model.

 **Patient**

patients[]

Field	Type	Example
id	string	P-00421
firstName , lastName	string	Sarah Johnson
email , phone	string	sarah.j@email.com · +1 (555) 234-5678
dob	ISO date string	1992-03-15
gender	string	Female / Male
address	string	456 Oak Ave, Chicago, IL
bloodGroup	string	A+, O-, AB-
doctor , department	string	Dr. Mike Chen · Cardiology
status	string	Stable In Treatment Discharged Admitted Critical
lastVisit	ISO date string	2026-06-29
visits , admissions , billed	number	12 · 3 · 4200
conditions	string[]	['Hypertension', 'Hyperlipidemia']
allergies	string[]	['Penicillin', 'Sulfa drugs']
vitals	object	{ bp, hr, temp, spo2 }

 **Doctor**

doctors[]

Field	Type	Example
id	string	D-001
name , specialty , department	string	Dr. Mike Chen · Cardiology
email , phone	string	mike.chen@medicore.com
experience	number (years)	15
patients	number	342
rating	number (0–5)	4.9
status	string	Available In Surgery On Leave
schedule	string	Mon–Fri 9AM–5PM

 Appointment

appointments[]

Field	Type	Example
id	string	A-1001
patientId , patient	string	P-00421 · Sarah Johnson
doctor , department	string	Dr. Mike Chen · Cardiology
date , time	string	2026-06-29 · 09:00
type	string	Checkup Consultation Follow-up
status	string	Confirmed Pending Checked In Cancelled

 Admission

admissions[]

Field	Type	Example
id	string	ADM-001
patientId , patient	string	P-00416 · Robert Lee
ward , room , bed	string	Cardiology Ward · C-204 · B2
admittedOn	ISO date string	2026-06-25
doctor , diagnosis	string	Dr. Mike Chen · Coronary Artery Disease
status	string	Admitted Critical

 Lab Order

labOrders[]

|

 Radiology Order

radiologyOrders[]

Field	Type	Example
id	string	LAB-001 / RAD-001
patient , patientId	string	Sarah Johnson · P-00421
test	string	CBC / Chest X-Ray / CT Angiography / MRI Brain
orderedBy	string	Dr. Mike Chen
date	ISO date string	2026-06-29
status	string	Pending In Progress Completed Scheduled
priority	string	Routine Urgent

 Pharmacy

pharmacy[]

--	--	--

Field	Type	Example
id	string	PH-001
medicine	string	Metoprolol 25mg
patient , patientId	string	Robert Lee · P-00416
prescribedBy	string	Dr. Mike Chen
date	ISO date string	2026-06-29
quantity	number	30
status	string	Dispensed Pending

 Inventory

inventory[]

Field	Type	Example
id	string	INV-001
name	string	Surgical Gloves (Box)
category	string	Consumables Fluids PPE Equipment
quantity , unit	number, string	450 · boxes
reorderLevel	number	100
status	string	In Stock Low Stock Out of Stock

 Billing

billing[]

Field	Type	Example
id	string	INV-B001
patient , patientId	string	Sarah Johnson · P-00421
amount , paid	number	1200 · 1200
status	string	Paid Partial Pending
date	ISO date string	2026-06-29
items	string[]	['Consultation', 'X-Ray']

 **Emergency** emergency[]

Field	Type	Example
id	string	EM-001
patient , patientId	string	David Kim · P-00418 (unknown patients have no patientId)
age	number (optional)	45 (for unknown patients)
condition	string	Acute Heart Failure
severity	string	Critical Urgent
arrivedAt	datetime string	2026-06-29 06:30
doctor , status	string	Dr. Anna White · Critical In Treatment

 **Message** messages[] |  **Notification** notifications[]

Field	Model	Example
<code>id</code>	Both	M-001 / NOT-001
<code>from</code> , <code>to</code> , <code>message</code>	Message	Dr. Lisa Park → Dr. Admin
<code>time</code>	Both	2026-06-29 08:30 / "10 min ago"
<code>read</code>	Both	boolean (false = unread)
<code>title</code> , <code>message</code> , <code>type</code>	Notification	Critical Patient Alert · danger info warning

15. Status & Badge Color Reference

All status values and their associated colors are defined centrally in `constants/colors.js` (app) and `data.js badgeForStatus()` (web).

Status Value	Context	App Color (bg / text)	Web CSS Class
Stable	Patient	#d1fae5 / #065f46	badge-success
In Treatment	Patient	#dbeafe / #1e40af	badge-warning
Discharged	Patient	#f3f4f6 / #374151	badge-gray
Admitted	Patient	#ede9fe / #5b21b6	badge-warning
Critical	Patient / Emergency / Admission	#fee2e2 / #991b1b	badge-danger
Available	Doctor / Room	#d1fae5 / #065f46	badge-success
In Surgery	Doctor	#fee2e2 / #991b1b	badge-warning
On Leave	Doctor	#fef3c7 / #92400e	badge-gray
Confirmed	Appointment	#d1fae5 / #065f46	badge-success
Pending	Appointment / Billing / Pharmacy	#fef3c7 / #92400e	badge-warning
Checked In	Appointment	#dbeafe / #1e40af	badge-gray
Cancelled	Appointment	#fee2e2 / #991b1b	badge-danger
Waiting	OPD Queue	#fef3c7 / #92400e	badge-warning
In Consultation	OPD Queue	#dbeafe / #1e40af	badge-info
Completed	OPD / Lab / Radiology	#d1fae5 / #065f46	badge-success
In Progress	Lab / Radiology	—	badge-info
Routine	Lab / Radiology	#ede9fe / #5b21b6	badge-primary
Urgent	Lab / Radiology	#fee2e2 / #991b1b	badge-danger
In Stock	Inventory	#d1fae5 / #065f46	badge-success
Low Stock	Inventory	#fef3c7 / #92400e	badge-warning
Out of Stock	Inventory	#fee2e2 / #991b1b	badge-danger
Paid	Billing	#d1fae5 / #065f46	badge-success
Partial	Billing	#fef3c7 / #92400e	badge-warning
Active	Department / Staff / User	#d1fae5 / #065f46	badge-success

Status Value	Context	App Color (bg / text)	Web CSS Class
Inactive	Department / Staff / User	#f3f4f6 / #374151	badge-gray
Dispensed	Pharmacy	#d1fae5 / #065f46	badge-success

16. Deployment & Setup Guide

Mobile App Setup

```
# Prerequisites: Node.js 18+, Expo CLI, Android Studio or Xcode (for simulators) # 1.
Install dependencies cd "hospital management system/app" npm install # 2. Start
development server npx expo start # 3. Platform-specific launch npx expo start --android
# Android emulator / device npx expo start --ios # iOS simulator (macOS only) npx expo
start --web # Web browser preview
```

Web Platform Setup

```
# No build step required – pure static HTML/CSS/JS # Option 1: Open directly in browser
open "hospital management system/website/index.html" # Option 2: Serve via local server
(recommended to avoid CORS) npx serve "hospital management system/website" # or python -
m http.server 8080 # then open http://localhost:8080
```

First-Time Login (Both Platforms)

⚠ On first launch, no registered users exist in the system. You must first register an account, or use the demo credentials. The demo accounts shown in Section 5 must be registered manually on first use — they are not pre-seeded into the registered users list.

Step	Action
1	Navigate to Register / Create Account
2	Enter name, email (e.g., <code>admin@medicore.com</code>), password (<code>admin123</code>), and role (Administrator)
3	Submit — you will be redirected to Login
4	Log in with the registered credentials
5	On first login, the seed data (SEED) is automatically loaded into storage

17. Known Limitations & Future Improvements

Current Limitations

#	Limitation	Impact
1	No real backend / database	All data is stored locally (AsyncStorage / localStorage). Data does not sync between devices or users. Each device/browser maintains its own isolated dataset.
2	No real authentication	Passwords are stored in plain text in localStorage/AsyncStorage. This is acceptable for a demo/prototype but must be replaced with hashed passwords and server-side auth before production use.
3	Web session ends on tab close	sessionStorage is cleared when the browser tab is closed, requiring re-login. Consider using localStorage with an expiry token for persistent web sessions.
4	No real-time updates	Data changes made by one user are not reflected for other users. There are no WebSocket or polling mechanisms. Each user sees only their own local state.
5	Storage size limits	AsyncStorage and localStorage have limits (~5–10 MB). Large datasets with images or attachments would exceed these limits.
6	Images loaded from Unsplash CDN	Dashboard hero, promo, and service card images are loaded from Unsplash URLs. These require an internet connection and may be unavailable offline or if the URLs change.
7	No PDF export implemented	The Reports screen mentions exportable PDFs, but no actual export function is implemented in the codebase. This would require a library like jsPDF or a server-side PDF renderer.
8	Dark mode incomplete (App)	The dark mode toggle is functional (toggle + persist), but most screens do not have dark-mode-specific styling applied. The preference is saved but visual theming is partial.
9	No input validation on forms	Add Patient, Add Appointment, and Register screens do minimal validation. Duplicate emails, malformed dates, or empty required fields are not consistently caught.
10	Web appointment form is non-functional	The landing page appointment form shows a toast confirmation but does not actually save the data or send a notification. It is a UI prototype only.

Recommended Future Improvements

- **Backend API** — Replace localStorage with a REST API (Node.js/Express + PostgreSQL or MongoDB) to enable multi-user real-time access
- **Authentication** — Implement JWT-based auth with bcrypt password hashing and secure HTTP-only cookies
- **Real-time updates** — Add WebSocket support (Socket.io) for live emergency alerts, OPD queue updates, and messaging
- **Push notifications** — Integrate Expo Notifications for critical patient alerts on mobile
- **PDF export** — Implement jsPDF or react-native-pdf for billing invoices and patient discharge summaries
- **Image uploads** — Add profile photo upload for patients and doctors using Expo ImagePicker + cloud storage

- **Dark mode** — Complete dark mode theming across all screens using the existing Colors.darkBg / Colors.darkCard tokens
- **Input validation** — Add Yup/Zod schema validation on all form inputs
- **Audit log** — Track all data changes (who changed what and when) for HIPAA compliance
- **Multi-language support** — Add i18n for regional hospital deployments

18. Design System — Color Palette

All colors are defined in `src/constants/colors.js` and used consistently across both platforms.

Token	Hex Value	Usage
<code>Colors.primary</code>	#2563eb	Primary buttons, active nav items, links
<code>Colors.primaryDark</code>	#1d4ed8	Hover state for primary elements
<code>Colors.secondary</code>	#0ea5e9	Secondary accents, tab bar active (#11B5E4 variant)
<code>Colors.success</code>	#10b981	Success states, Stable/Available badges
<code>Colors.warning</code>	#f59e0b	Pending, Low Stock, On Leave warnings
<code>Colors.danger</code>	#ef4444	Critical, Cancelled, emergency badge, notification dot
<code>Colors.info</code>	#6366f1	Info notifications, In Progress states
<code>Colors.dark</code>	#0f172a	Sidebar background, dark header
<code>Colors.gray900-50</code>	—	Full Tailwind-compatible gray scale for text, borders, backgrounds
<code>Colors.darkCard</code>	#1e293b	Card background in dark mode

19. Complete File Structure

Mobile App

```

app/ └─ App.js ← Root component: GestureHandler + AppProvider + Navigator └─ app.json
    ← Expo config (name, slug, icon, splash) └─ babel.config.js ← Babel preset-expo config
    └─ package.json ← Dependencies (Expo 55, React 19, RN 0.83.6) └─ src/ └─ components/
    │ └─ Avatar.js │ └─ Card.js │ └─ EmptyState.js │ └─ PageHeader.js │ └─
    ScreenHeader.js │ └─ SearchBar.js │ └─ StatCard.js │ └─ StatusBadge.js └─ constants/
    │ └─ colors.js ← Colors + StatusColors maps └─ context/ │ └─ AppContext.js ← Global
    state: useReducer + AsyncStorage └─ data/ │ └─ seedData.js ← SEED dataset +
    DEMO_CREDENTIALS └─ navigation/ │ └─ AppNavigator.js ← RootStack → MainDrawer →
    HomeTabs └─ screens/ └─ LoginScreen.js └─ RegisterScreen.js └─ DashboardScreen.js
    └─ PatientsScreen.js └─ PatientProfileScreen.js └─ AddPatientScreen.js └─
    DoctorsScreen.js └─ DoctorProfileScreen.js └─ AppointmentsScreen.js └─
    AddAppointmentScreen.js └─ AdmissionsScreen.js └─ OPDQueueScreen.js └─
    DischargeScreen.js └─ MedicalRecordsScreen.js └─ NursingScreen.js └─
    LaboratoryScreen.js └─ RadiologyScreen.js └─ PharmacyScreen.js └─ RoomsScreen.js └─
    EmergencyScreen.js └─ DepartmentsScreen.js └─ StaffScreen.js └─ InventoryScreen.js
    └─ BillingScreen.js └─ ReportsScreen.js └─ MessagesScreen.js └─
    NotificationsScreen.js └─ UsersScreen.js └─ SettingsScreen.js
  
```

Web Platform

```

website/ └─ index.html ← Public landing page └─ css/ │ └─ landing.css ← Landing page
    styles (animations, hero, sections) │ └─ style.css ← Shared dashboard layout: sidebar,
    cards, tables, badges └─ js/ │ └─ data.js ← HMSData singleton (localStorage CRUD +
    utilities) │ └─ nav.js ← Sidebar renderer + RBAC page guard │ └─ app.js ← Auth
    helpers, toast notifications, shared utilities │ └─ main.js ← Landing page: counters,
    FAQ, particles, scroll effects └─ pages/ └─ login.html ← Auth └─ register.html ← Auth
    └─ dashboard.html ← Main └─ patients.html ← Main └─ patient-profile.html ← Main └─
    doctors.html ← Main └─ doctor-profile.html ← Main └─ departments.html ← Main └─
    appointments.html ← Main └─ add-appointment.html ← Main └─ opd-queue.html ← Main └─
    admissions.html ← Clinical └─ discharge.html ← Clinical └─ medical-records.html ←
    Clinical └─ nursing.html ← Clinical └─ laboratory.html ← Clinical └─ radiology.html ←
    Clinical └─ pharmacy.html ← Clinical └─ rooms.html ← Clinical └─ emergency.html ←
    Clinical └─ staff.html ← Administration └─ inventory.html ← Administration └─
    billing.html ← Administration └─ reports.html ← Administration └─ users.html ←
  
```

```
Administration (Admin only) |— messages.html ← Administration |— notifications.html ←  
Administration |— settings.html ← Administration |— my-profile.html ← Administration
```



MediCore HMS

Hospital Management System — Technical Documentation

2
Platforms

28
Screens / Pages

16
Data Models

4
User Roles

19
Doc Sections

Version 1.0.0 · MediCore General Hospital · July 2026